🚀 LECTURE 13 — BUBBLE SORT, INSERTION SORT & BINARY SEARCH

DSA ka asli matlab: sochna, observe karna, aur decide karna

🎯 CHAPTER VISION — IS LECTURE KA ASLI MAQSAD

Is lecture ka goal sirf 3 algorithms sikhana nahi hai.
Asli goal hai tumhe ye sikhana ki:

- 🧠 **Array ke andar movement kaise hoti hai**
- 🔍 **Loop chalne ka matlab kya hota hai**
- ⚖️ **Best vs Worst case kyu banta hai**
- 🧩 **Kab kaunsa algorithm use karna chahiye**

👉 Ye lecture samajh gaya =
Sorting + Searching foundation permanently strong

🧠 ARRAY — BASE CLEAR (VERY IMPORTANT)

Array ka matlab:

> Same type ke elements ka **continuous memory block**

Example:

```
Index:   0   1   2   3   4
Array:  [5,  1,  4,  2,  8]
```

📌 **Important Observation**

- Array me element shift karna = **costly**
- Isliye sorting algorithms ko carefully design kiya jaata hai

---

🔁 INSERTION SORT :

📌 Insertion Sort kya karta hai?

Insertion sort ka mindset:

> “Left side already sorted hai,
> mujhe bas naye element ko usme ghusana hai.”

🧠 FIRST-PRINCIPLE THINKING

Question:

> “Agar mere paas sorted list hai,
> to naya element sahi jagah kaise pahunchau?”

Answer:

- Peeche se compare karo
- Jab tak chhota element mile, **swap karte jao**
- Jaise hi sahi jagah mile → **ruk jao**

👉 Yehi `break` ka real kaam hai

👀 ARRAY VISUAL (DEEP WALKTHROUGH)

Initial:

```
[8, 3, 5, 2]
```

i = 1

```
3 < 8 → swap
```

```
[3, 8, 5, 2]
```

i = 2

```
5 < 8 → swap
```

```
[3, 5, 8, 2]
```

i = 3

`2 < 8 → swap → [3, 5, 2, 8]`

`2 < 5 → swap → [3, 2, 5, 8]`

`2 < 3 → swap → [2, 3, 5, 8]`

📌 **Invariant (INTERVIEW GOLD)**
👉 Har iteration ke baad **left part sorted hota hai**

💻 Insertion Sort — FINAL (LeetCode Ready, Tumhara Code)

```
class Solution {

public:

    vector<int> insertionSort(vector<int>& arr) {

        int n = arr.size();


        // i current element ko sorted part me insert karega

        for (int i = 0; i < n; i++) {


            // j peeche jaakar correct position dhundhega

            for (int j = i; j > 0; j--) {


                // Agar current element chhota hai previous se

                if (arr[j] < arr[j - 1]) {

                    swap(arr[j], arr[j - 1]); // left shift

                }

                else {

                    break; // already correct position mil gayi

                }

            }
```

```
        }
        return arr;
    }
};
```

⏱️ Insertion Sort — TC / SC (KYU?)

🔴 Worst Case — `O(n²)`

- Reverse sorted array
- Har element poore left part se compare karega

🟢 Best Case — `O(n)`

- Already sorted
- `break` turant lag jaata hai

💾 Space Complexity — `O(1)`

- Koi extra array nahi
- Sirf indices use ho rahe hain

📌 **Stable Sort**
Equal elements ka order change nahi hota ✓

🌍 Real-Life Mapping (Insertion Sort)

🃏 Cards haath me lagana

- Har naya card peeche check hota hai
- Jahan fit ho → wahi insert

👉 Isliye insertion sort:

- Small data

- Nearly sorted data
  me **best performer** hai

🔁 BUBBLE SORT :

📌 Bubble Sort ka real kaam

Bubble sort ka logic:

> “Sabse bada element dheere-dheere end me pahunchao”

🧠 FIRST-PRINCIPLE SOCH

Question:

> “Largest element ko last me kaise le jaaun bina direct jump ke?”

Answer:

- Neighbours compare karo
- Bada element ko right push karo

👀 ARRAY VISUAL

```
[5, 1, 4, 2, 8]
```

After first pass:

```
[1, 4, 2, 5, 8]
```

📌 **Invariant**
👉 Har pass ke baad **last element sorted hota hai**

💻 Bubble Sort — Code part :

```
class Solution {

public:

    vector<int> bubbleSort(vector<int>& arr) {

        int n = arr.size();


        for (int i = 0; i < n; i++) {


            bool swapped = false; // optimization flag


            for (int j = 0; j < n - 1; j++) {

                if (arr[j] > arr[j + 1]) {

                    swap(arr[j], arr[j + 1]);

                    swapped = true;

                }

            }


            // Agar ek bhi swap nahi hua → already sorted

            if (!swapped)

                return arr;

        }

        return arr;

    }

};
```

⏱️ Bubble Sort — TC / SC (DETAIL)

🔴 Worst Case — $O(n^2)$

- Reverse sorted array

🟢 Best Case — $O(n)$

- Already sorted + `swapped` flag

💾 Space Complexity — $O(1)$

- In-place algorithm

📌 **Stable Sort ✓**

🌍 Real-Life Mapping (Bubble Sort)

👨‍🏫 Height-wise line:

- Lamba banda dheere-dheere peeche jaata hai
- Har round ke baad ek banda fix

🔍 BINARY SEARCH (START–END APPROACH)

📌 Binary Search ka real idea

Binary search ka logic:

> “Sorted data hai, to aadha kyu na kaat dein?”

🧠 FIRST-PRINCIPLE THINKING

Question:

> “Har element check karna wasteful nahi?”

Answer:

- Middle se decision lo
- Half discard karo

👀 ARRAY VISUAL

```
[2, 4, 6, 8, 10, 12, 14]
```

Target = `10`

Steps:

- mid = 8 → right
- mid = 12 → left
- mid = 10 → FOUND

💻 Binary Search — FINAL (LeetCode Ready Code)

```
class Solution {

public:

    int search(vector<int>& arr, int target) {

        int start = 0;

        int end = arr.size() - 1;


        while (start <= end) {


            // overflow-safe mid

            int mid = start + (end - start) / 2;


            if (arr[mid] == target) {
```

```
            return mid;
        }
        else if (arr[mid] < target) {
            start = mid + 1; // right half
        }
        else {
            end = mid - 1;   // left half
        }
    }
    return -1;
  }
};
```

⏱️ Binary Search — TC / SC (WHY?)

⏱️ Time — `O(log n)`

- Har step me size aadha
- n → n/2 → n/4 → …

💾 Space — `O(1)`

- Sirf pointers use ho rahe hain

⚠️ Precondition

- Array **sorted hona mandatory**

🌍 Real-Life Mapping (Binary Search)

📖 Dictionary:

- Page 1 se nahi
- Beech se shuru
- Har baar aadhi book skip

🌟 FINAL BIG PICTURE (YAAD RAKHNA)

- Bubble Sort → **comparison samajhata hai**
- Insertion Sort → **placement samajhata hai**
- Binary Search → **decision making sikhata hai**

👉 DSA ka asli matlab:

Problem ko todna, observation banana, best choice lena

📚 ULTIMATE SUMMARY TABLE (Quick Comparison)

🧩 Algorithm	⚡ Best TC	🔥 Worst TC	💾 Space	🔁 Stable
🫧 **Bubble Sort**	O(n)	O(n²)	O(1)	✅ Yes
✍️ **Insertion Sort**	O(n)	O(n²)	O(1)	✅ Yes
🔍 **Binary Search**	O(log n)	O(log n)	O(1)	➖ N/A